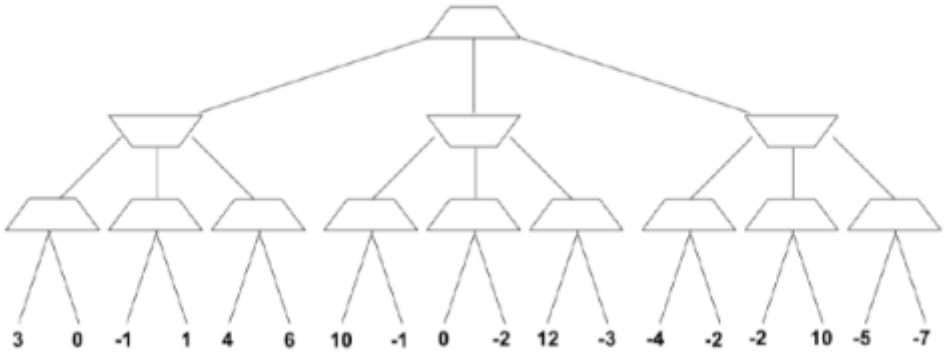
BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Tạ Quang Thắng
MSSV	3123410345

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây

- **Tầng 2 (MAX - lấy giá trị lớn nhất từ các cặp lá con):**
  - Nút 1: $max(3,0) = 3$
  - Nút 2: $max(-1,1) = 1$
  - Nút 3: $max(4,6) = 6$
  - Nút 4: $max(10,-1) = 10$
  - Nút 5: $max(0,-2) = 0$
  - Nút 6: $max(12,-3) = 12$
  - Nút 7: $max(-4,-2) = -2$
  - Nút 8: $max(-2,10) = 10$
  - Nút 9: $max(-5,-7) = -5$
- **Tầng 1 (MIN - mỗi nút nhận 3 con từ Tầng 2, chọn giá trị nhỏ nhất):**
  - Con 1 (nhánh trái): $min(3,1,6) = 1$
  - Con 2 (nhánh giữa): $min(10,0,12) = 0$
  - Con 3 (nhánh phải): $min(-2,10,-5) = -5$
- **Tầng 0 (MAX - Gốc):**

- Gốc: $max(1,0,-5) = 1$

b) Lựa chọn của MAX và giá trị trò chơi

- **Nhánh MAX nên chọn:** MAX nên chọn nhánh bên trái (dẫn đến giá trị 1).
- **Giá trị của trò chơi:** 1

c) Quá trình cắt tỉa Alpha-Beta Pruning (duyệt từ trái sang phải)

Khởi tạo tại Gốc: $\alpha = -\infty, \beta = +\infty$.

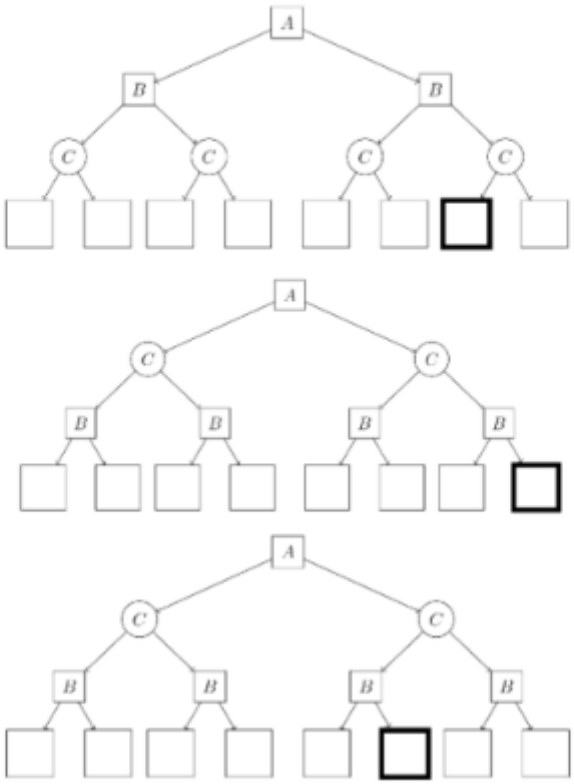
1. **Nhánh con 1 của Gốc (MIN):** $\alpha = -\infty, \beta = +\infty$
   - Nút MAX 1: Xét 3, cập nhật $\alpha = 3$; xét 0 → Giá trị = 3. Cập nhật tại nút MIN: $\beta = min(+\infty, 3) = 3$.
   - Nút MAX 2: $\alpha = -\infty, \beta = 3$. Xét -1; xét 1 → Giá trị = 1. Cập nhật tại nút MIN: $\beta = min(3,1) = 1$.
   - Nút MAX 3: $\alpha = -\infty, \beta = 1$. Xét lá đầu tiên có giá trị là 4. Vì $4 \geq \beta$ ($4 \geq 1$), xảy ra cắt tỉa α. Bỏ qua lá 6. Giá trị nút MAX 3 ≥ 4.
   - Hoàn thành nhánh 1 của Gốc, giá trị trả về là 1. Cập nhật tại Gốc: $\alpha = max(-\infty, 1) = 1, \beta = +\infty$.
2. **Nhánh con 2 của Gốc (MIN):** Kế thừa $\alpha = 1, \beta = +\infty$
   - Nút MAX 4: $\alpha = 1, \beta = +\infty$. Xét 10; xét -1 → Giá trị = 10. Cập nhật tại nút MIN: $\beta = min(+\infty, 10) = 10$.
   - Nút MAX 5: $\alpha = 1, \beta = 10$. Xét 0; xét -2 → Giá trị = 0. Cập nhật tại nút MIN: $\beta = min(10,0) = 0$.
   - Lúc này tại nút MIN, $\beta = 0 \leq \alpha = 1$ → Xảy ra cắt tỉa β ngay tại nút MIN. Toàn bộ nút MAX 6 bị cắt tỉa. Bỏ qua cả hai lá 12 và -3.
3. **Nhánh con 3 của Gốc (MIN):** $\alpha = 1, \beta = +\infty$
   - Nút MAX 7: $\alpha = 1, \beta = +\infty$. Xét -4; xét -2 → Giá trị = -2. Cập nhật tại nút MIN: $\beta = min(+\infty, -2) = -2$.
   - Lúc này tại nút MIN, $\beta = -2 \leq \alpha = 1$ → Xảy ra **cắt tỉa** β ngay tại nút MIN. Toàn bộ các nhánh con còn lại (nút MAX 8 và MAX 9) bị cắt tỉa. Bỏ qua các lá: **-2**, **10**, **-5**, **-7**.

d) Thống kê số nút lá

- **Các nút lá không cần thăm (bị cắt tỉa):** 6, 12, -3, -2, 10, -5, -7 (tổng cộng 7 lá).
- So sánh số nút lá được đánh giá:
  - Thuật toán Minimax: Phải duyệt tất cả 18 nút lá.
  - Thuật toán Alpha-Beta Pruning: Chỉ cần duyệt 11 nút lá (tiết kiệm được gần 39% số phép duyệt lá).

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.



Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a) Trường hợp 1 (Cắt tỉa tại lá thứ 3 của nút chance C bên phải)

- **Khả năng:** Có thể cắt tỉa nếu có chặn trên/chặn dưới hữu hạn của hàm lượng giá.
- **Gán giá trị ví dụ:**
  - Giả sử giá trị các lá thuộc đoạn $[0,10]$.
  - Nhánh trái của A (nút B bên trái) tính được giá trị: $v_L = 2$.
  - Tại nút B bên phải, nhánh C thứ nhất cho kỳ vọng $E_1 = 3$. Vì B là MAX nên $v(B) \geq 3 > 2$.
  - Vì A là MIN và đã có nhánh trái bằng 2, mà nhánh phải $v(B) \geq 3$, nút A chắc chắn sẽ không bao giờ chọn nhánh bên phải nữa.
  - **Điều kiện cắt:** Ngay khi xác định được một nhánh của MAX B có giá trị vượt quá giá trị hiện tại của MIN A, ta có thể cắt toàn bộ các nhánh còn lại của B mà không cần duyệt đến lá viền đậm.

b) Trường hợp 2 (Cắt tỉa lá thứ 4 của nút B thứ hai dưới chance node C bên phải)

- **Khả năng:** Không thể cắt tỉa (nếu không có chặn giá trị) hoặc Có thể nếu biết khoảng chặn $[V_{min}, V_{max}]$.
- **Giải thích:**
  - Nút C tính giá trị theo trung bình có trọng số: $V(C) = 0.5 \cdot V(B_1) + 0.5 \cdot V(B_2)$.
  - Dù $V(B_1)$ đã biết, ta vẫn cần biết $V(B_2)$ để tính kỳ vọng. Nếu không có chặn trên/chặn dưới hữu hạn, $V(B_2)$ có thể tiến tới $-\infty$ hoặc $+\infty$, làm thay đổi hoàn toàn giá trị kỳ vọng của C. Do đó trong trường hợp tổng quát không có cận biên, không thể cắt tỉa.

c) Trường hợp 3 (Cắt tỉa lá thứ 3 thuộc nút B thứ nhất dưới chance node C bên phải)

- **Khả năng:** Tương tự trường hợp 2, Không thể cắt tỉa nếu không có cận giá trị lá, hoặc Có thể cắt tỉa nếu biết chặn giá trị $[V_{min}, V_{max}]$.
- **Điều kiện cắt khi có cận $[V_{min}, V_{max}]$:**
  - Giả sử lá nằm trong $[0,10]$. Nhánh C bên trái cho kết quả $V(C_1) = 1$.
  - Tại nhánh C bên phải, nếu nhánh con thứ nhất B_1 đã có một lá mang giá trị 10, thì $V(B_1) = \max(10, \dots) \ge 10$.
  - Khi đó giá trị kỳ vọng nhỏ nhất có thể của C_2 là: $V(C_2) \geq 0.5 \cdot 10 + 0.5 \cdot 0 = 5$.
  - Vì A là nút MIN: A sẽ chọn $min(V(C_1), V(C_2)) = min(1, \geq 5) = 1$. Giá trị của C_2 chắc chắn luôn lớn hơn 1, do đó nhánh còn lại có thể cắt tỉa.

d) Vì sao cắt tỉa trong cây Expectimax khác với Alpha-Beta trong cây Minimax thông thường?

- Trong cây Minimax thuần túy, các phép toán tại các nút là min hoặc max. Các phép toán này mang tính chất đơn điệu cục bộ: một giá trị lá xấu/tốt có thể lập tức đặt cận chắc chắn cho toàn bộ nút cha.
- Trong cây có nút ngẫu nhiên (Expectimax), nút chance tính theo tổ hợp tuyến tính (trung bình có trọng số). Một kết quả cực kỳ lớn hoặc cực kỳ nhỏ ở lá chưa xét vẫn có thể kéo giá trị trung bình lên hoặc xuống, trừ khi ta biết trước khoảng giá trị chặn trên/chặn dưới toàn cục của toàn bộ trò chơi.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có $V_M \leq V_E$

- Đáp án: Đúng
- Giải thích: Minimax giả định đối thủ chọn nước đi tồi tệ nhất đối với ta (min). Trong khi đó, Expectimax tính theo kỳ vọng của một đối thủ ngẫu nhiên (trung bình trọng số). Vì giá trị nhỏ nhất của một tập luôn nhỏ hơn hoặc bằng giá trị trung bình của tập đó ($\min_i x_i \leq \sum p_i x_i$), nên giá trị mà MAX thu được khi đối đầu với đối thủ ngẫu nhiên luôn ≥ khi đối đầu với đối thủ tối ưu.

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M

- Đáp án: Đúng
- Giải thích: Chính sách Minimax đảm bảo rằng dù đối phương đánh kiểu gì (kể cả ngẫu nhiên hay tối ưu), lợi ích nhận được trong trường hợp xấu nhất vẫn ít nhất bằng V_M. Vì kỳ vọng là trung bình các kết quả, và mỗi kết quả đều ≥ trường hợp xấu nhất trên cây Minimax tương ứng, nên lợi ích kỳ vọng chắc chắn ≥ V_M.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E

- Đáp án:Sai
- Giải thích: V_E chỉ đạt được khi thực thi chính sách tối ưu của Expectimax. Chính sách Minimax quá thận trọng (chỉ phòng thủ trường hợp xấu nhất), do đó lợi ích kỳ vọng đạt được khi dùng chiến thuật Minimax thường thấp hơn V_E.

Xét trò chơi một người, kết quả hành động không xác định trước (+1 thắng, -1 thua)

- **Các phương án có thể phù hợp:**
  - [x] Expectimax với tiện ích trạng thái cuối $+1/-1$
  - [x] Value Iteration với phần thưởng 0 ở trạng thái trung gian và $+1/-1$ ở kết quả thắng/thua

d) Giải thích:

- Expectimax: Phù hợp nếu cây trò chơi là hữu hạn (DAG không có chu trình), người chơi đóng vai trò MAX và môi trường ngẫu nhiên đóng vai trò chance nodes với xác suất chuyển đổi đã biết.

- Value Iteration (MDP): Phù hợp tổng quát hơn, đặc biệt khi trò chơi có thể có chu trình (vòng lặp trạng thái), không gian trạng thái vô hạn hoặc có thể quay lại trạng thái cũ.
- Minimax: Không phù hợp vì đây là trò chơi ngẫu nhiên một người, không có đối thủ có chủ đích đối kháng tối ưu.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

6 biến: A, B, C, D, E, F. Ràng buộc nhị phân
$(A, B), (A, C), (A, D), (B, C), (B, E), (C, F)$.

a) Đồ thị ràng buộc

- **Các đỉnh:** $\{A, B, C, D, E, F\}$
- **Các cạnh:**
  - Nối A với: B, C, D
  - Nối B với: A, C, E
  - Nối C với: A, B, F
  - D chỉ nối với A
  - E chỉ nối với B
  - F chỉ nối với C

b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

- **Trả lời:** B, C, D
- **Giải thích:** Forward Checking chỉ kiểm tra và lọc miền giá trị của các biến chưa gán có ràng buộc trực tiếp (kề) với biến vừa được gán (A).

c) Sau khi gán A và FC, tiếp tục gán B: miền của những biến nào có thể bị thay đổi bởi Forward Checking tại B?

- **Trả lời:** C, E
- **Giải thích:** Khi gán B, Forward Checking xét các láng giềng chưa gán của B. Các láng giềng của B là A, C, E; vì A đã được gán trước đó, nên chỉ có C và E được kiểm tra và có thể bị thu hẹp miền giá trị.

d) Sau khi gán A, nếu duy trì Arc Consistency (MAC), miền của những biến nào có thể thay đổi?

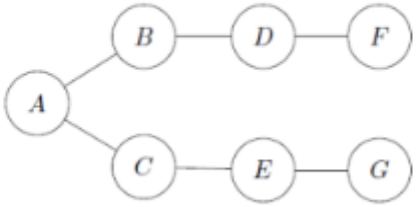
- **Trả lời:** B, C, D, E, F (tất cả các biến còn lại)
- **Giải thích:** Khi A được gán, miền của B, C, D bị thu hẹp. Sự thay đổi miền tại B có thể kích hoạt lan truyền làm thay đổi miền của E (qua cung $E \rightarrow B$). Tương tự, sự thay đổi miền tại C có thể làm thay đổi miền của F (qua cung $F \rightarrow C$).

e) Sau khi gán A và MAC, tiếp tục gán B và duy trì MAC: miền của những biến nào có thể thay đổi?

- **Trả lời:** C, E, F
- **Giải thích:** Gán B trực tiếp làm thay đổi miền láng giềng chưa gán là C và E. Sau đó, nếu miền của C tiếp tục bị thu hẹp, nó có thể lan truyền qua cung $F \rightarrow$

C làm thu hẹp tiếp miền của F. (Miền của D chỉ phụ thuộc A nên không bị ảnh hưởng thêm bởi B).

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến (nguồn: rec3.pdf, trang 6)

f) Thứ tự $A \to B \to C \to D \to E \to F \to G$:

- Gán 1 biến (A): Chưa chạy AC.
- Gán 2 biến (A, B): Đã gán số chẵn → Chạy AC. Vì đồ thị là dạng cây và AC được duy trì trên các biến còn lại, mọi xung đột dẫn đến vô nghiệm ở các bước sau sẽ được phát hiện ngay lập tức mà không cần thử sai sâu.
- **Số lần backtrack xấu nhất:** $O(d)$ (hoặc tối đa $d - 1$ lần tại biến đầu nhánh).

g) Thứ tự $G \to E \to C \to A \to B \to D \to F$:

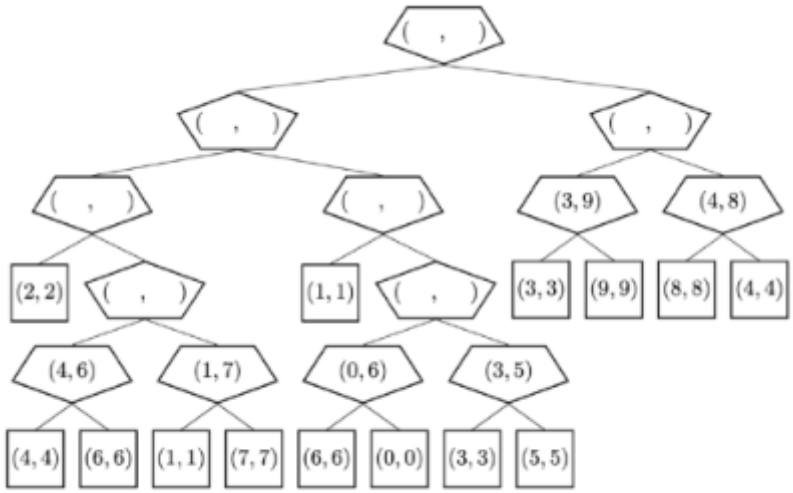
- Thứ tự đi dọc theo chuỗi nhánh $G - E - C$, qua gốc A, rồi sang nhánh còn lại.
- Sau các bước gán thứ 2 (E), thứ 4 (A), thứ 6 (D), AC đều được kích hoạt để dọn dẹp các giá trị không hợp lệ của các biến phía sau.
- **Số lần backtrack xấu nhất:** Bị giới hạn ở bậc đa thức, tối đa tỉ lệ với số giá trị thử sai của các biến lẻ không chạy AC: $O(d)$.

h) Thứ tự $G \to F \to E \to D \to C \to B \to A$:

- Đây là thứ tự duyệt ngược từ các nút lá ở hai nhánh khác nhau hướng về gốc (G và F là hai lá độc lập, không có cạnh trực tiếp).
- Việc gán các lá độc lập trước khi gán các nút tổ tiên (C, B, A) khiến các bước lẻ có thể gán các giá trị không tương thích với nghiệm toàn cục.
- **Số lần backtrack xấu nhất:** Trong trường hợp xấu nhất có thể lên tới cấp số mũ đối với các lựa chọn lá rời rạc: $O(d^2)$ đến $O(d^3)$.

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Hình 4. Cây trò chơi Pacman–Casper

a) Điền tất cả các cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper.

- **Tầng 3 (Casper đưa ra hành vi thực tế, nhưng Pacman nghĩ Casper là MIN):**
  - Nhánh 1: Các lá con là $(4,4)$ và $(6,6)$.
    - Pacman nghĩ Casper chọn con có tiện ích nhỏ nhất → $u = min(4,6) = 4$.
    - Thực tế Casper chọn con có tiện ích lớn nhất → $v = max(4,6) = 6$.
    - Điền: $(\mathbf{4,6})$ (Đã có sẵn trên hình).
  - Nhánh 2: Các lá con là $(1,1)$ và $(7,7)$.
    - $u = min(1,7) = 1$; $v = max(1,7) = 7$ → Điền: $(\mathbf{1,7})$ (Đã có sẵn trên hình).
  - Nhánh 3: Các lá con là $(6,6)$ và $(0,0)$.
    - $u = min(6,0) = 0$; $v = max(6,0) = 6$ → Điền: $(\mathbf{0,6})$ (Đã có sẵn trên hình).
  - Nhánh 4: Các lá con là $(3,3)$ và $(5,5)$.
    - $u = min(3,5) = 3$; $v = max(3,5) = 5$ → Điền: $(\mathbf{3,5})$ (Đã có sẵn trên hình).

- **Tầng 2 (Pacman là MAX - Pacman hành động dựa trên việc tối đa hóa giá trị tin tưởng u):**
    - Nút bên trái nhất: Con là $(2,2)$ và ô trống. Ô trống này nhận từ nút con bên dưới là $(4,6)$ và $(1,7)$.
        - Ô trống tầng này: Pacman chọn con có u lớn hơn giữa $(4,6)$ và $(1,7)$ → chọn nhánh $(4,6)$. Do đó giá trị truyền lên là $(\boldsymbol{4,6})$.
        - Nút cha phía trên (trái ngoài cùng tầng 2): Nhận con là $(2,2)$ và $(4,6)$. Pacman chọn nhánh có u lớn hơn → $\max_u(2,4) = 4$ (chọn nhánh $(4,6)$). Giá trị là $(\boldsymbol{4,6})$.
    - Nút thứ hai tầng 2: Nhận $(1,1)$ và ô trống bên cạnh.
        - Ô trống nhận từ $(0,6)$ và $(3,5)$ → Pacman chọn theo $\max_u(0,3) = 3$ → lấy cặp $(\boldsymbol{3,5})$.
        - Nút cha: Nhận $(1,1)$ và $(3,5)$ → Pacman chọn $\max_u(1,3) = 3$ → lấy cặp $(\boldsymbol{3,5})$.
    - Các nút bên phải:
        - Nhánh có lá $(3,3)$ và $(9,9)$ → Casper thực tế chọn $(9,9)$, Pacman nghĩ chọn $(3,3)$ → cặp $(\boldsymbol{3,9})$ (Đã có trên hình).
        - Nhánh có lá $(8,8)$ và $(4,4)$ → Casper thực tế chọn $(8,8)$, Pacman nghĩ chọn $(4,4)$ → cặp $(\boldsymbol{4,8})$ (Đã có trên hình).
- **Tầng 1 (Casper):**
    - Nút bên trái: Nhận hai con từ tầng 2 là $(4,6)$ và $(3,5)$.
        - Pacman nghĩ Casper chọn $\min_u(4,3) = 3$ (nhánh phải).
        - Nhưng Casper thực tế chọn nhánh tối đa hóa v: $\max_v(6,5) = 6$ (nhánh trái).
        - Giá trị truyền lên: $(\boldsymbol{3,6})$.
    - Nút bên phải: Nhận hai con là $(3,9)$ và $(4,8)$.
        - Pacman nghĩ Casper chọn $\min_u(3,4) = 3$ (nhánh trái).
        - Casper thực tế chọn $\max_v(9,8) = 9$ (nhánh trái).
        - Giá trị truyền lên: $(\boldsymbol{3,9})$.
- **Gốc (Pacman):**
    - Pacman so sánh hai nhánh con có giá trị u là 3 và 3. Nếu quy ước hòa chọn nhánh trái, Pacman đi sang trái mang lại giá trị thực tế là 6; nếu chọn nhánh phải mang lại giá trị thực tế là 9.
- **Giải thích sự khác biệt giữa u và v tại nút Casper:**
    - u phản ánh niềm tin sai lệch của Pacman: Pacman tính toán giá trị này dựa trên giả định đối thủ là kẻ thù cố tình chọn nước đi có tiện ích nhỏ nhất (min).

- v là thực tế khách quan: Casper thực tế lại là bạn và chủ động chọn hành động mang lại lợi ích cao nhất cho Pacman (max).

b) Cắt tỉa khi duyệt từ trái sang phải

- Vì Casper đưa ra quyết định dựa trên việc tối đa hóa giá trị thực tế v, nhưng Pacman đưa ra quyết định dựa trên u, nên không thể dùng giá trị cận $\alpha - \beta$ thông thường một cách mù quáng.
- Tại nút Casper bên phải, khi đã duyệt xong nhánh con thứ nhất cho kết quả $(3,9)$, nếu biết chắc giá trị tiện ích tối đa của trò chơi là 9, Casper biết nhánh sau không thể có $v > 9$, có thể cắt tỉa nhánh con $(4,8)$ và các lá của nó.

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

- Cả Pacman và Casper đều trở thành các nút MAX (hợp tác để tối đa hóa tiện ích).
- Cây trò chơi chuyển thành bài toán tìm giá trị lớn nhất trên toàn bộ các lá: $max(tất cả các lá) = 9$.
- Trò chơi sẽ đạt giá trị cực đại là 9.

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Thuật toán	**Điều kiện đầy đủ**	**Điều kiện tối ưu**
Breadth-First Search (BFS)	Đầy đủ nếu hệ số phân nhánh $b < \infty$. (Trên đồ thị cần kiểm tra trạng thái trùng lặp để tránh chu trình vô hạn).	Tối ưu nếu chi phí của mọi bước đi (cạnh) là như nhau (bằng một hằng số dương $c > 0$).
Depth-First Search (DFS)	**Tìm kiếm trên cây:** Không đầy đủ nếu không gian trạng thái có độ sâu vô hạn hoặc có chu trình. **Tìm kiếm trên đồ thị:** Đầy đủ nếu không gian trạng thái là hữu hạn.	Không tối ưu (có thể trả về nghiệm sâu hơn trước khi tìm thấy nghiệm tối ưu).
Iterative Deepening Search (IDS)	Đầy đủ nếu hệ số phân nhánh $b < \infty$	Tối ưu nếu chi phí của mọi bước đi là như nhau.
A* Search	Đầy đủ nếu chi phí mỗi bước đi $\geq \epsilon > 0$ và hệ số phân nhánh $b < \infty$.	**Trên cây:** Tối ưu khi hàm heuristic $h(n)$ là chấp nhận được (admissible - $0 \leq h(n) \leq h^*(n)$). **Trên đồ thị:** Tối ưu khi $h(n)$ là nhất quán (consistent / monotonic)

Giải thích ảnh hưởng của điều kiện chi phí bước và hàm heuristic:

- Chi phí bước $\epsilon > 0$: Ngăn chặn hiện tượng đường đi vô hạn với tổng chi phí hữu hạn (Zeno's paradox), đảm bảo thuật toán không bị sa lầy vào việc mở rộng vô tận các đường đi có chi phí cực nhỏ.
- Heuristic chấp nhận được: Đảm bảo không bao giờ ước lượng vượt quá chi phí thực tế, giúp đường đi đến đích tối ưu luôn có $f(n)$ không bị thổi phồng.
- Heuristic nhất quán: Đảm bảo giá trị hàm $f(n) = g(n) + h(n)$ không giảm dọc theo đường đi, từ đó trạng thái khi được lấy ra khỏi danh sách PriorityQueue (mở rộng) chắc chắn đã đạt chi phí đường đi ngắn nhất đến nó.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a) Chi phí tăng n; cạnh $X \to Y$ thuộc đường tối ưu cũ và đã được khám phá

- **Chi phí tối ưu có thay đổi không:** Có thể thay đổi (tăng thêm $k \leq n$, hoặc giữ nguyên chi phí nếu có một đường đi thay thế khác có chi phí bằng C^*).
- **Cách cập nhật:** Không cần chạy lại từ đầu. Tăng chi phí $g(Y)$ và các nút con kế thừa từ Y thêm n. Cập nhật lại giá trị trong frontier/open list và tiếp tục chạy A* từ frontier này cho đến khi lấy được nút đích mới.

b) Chi phí giảm n; cạnh $X \to Y$ thuộc đường tối ưu cũ và đã được khám phá

- **Chi phí tối ưu có thay đổi không:** Chắc chắn thay đổi, giảm đúng bằng n. Chi phí mới là $C^* - n$.
- **Cách cập nhật:** Đường đi tối ưu cũ vẫn là đường đi tối ưu mới. Chỉ cần cập nhật giảm chi phí các nút từ Y đến đích thêm n, không cần chạy lại A*.

c) Chi phí tăng n; cạnh $X \to Y$ không thuộc đường tối ưu cũ nhưng đã được khám phá

- **Chi phí tối ưu có thay đổi không: Không thay đổi**.
- **Cách cập nhật:** Đường đi $P^*_{cũ}$ không đi qua cạnh này và chi phí của nó không đổi. Tăng chi phí của một cạnh không tối ưu chỉ làm cho nó kém hấp dẫn hơn, do đó đường đi tối ưu cũ vẫn giữ nguyên vị trí tối ưu.

d) Chi phí giảm n; cạnh $X \to Y$ không thuộc đường tối ưu cũ nhưng đã được khám phá

- **Chi phí tối ưu có thay đổi không:** Có thể thay đổi (nếu đường đi mới qua $X \to Y$ có chi phí $< C^*$).
- **Cách cập nhật:** Cập nhật lại $g(Y) = min(g(Y), g(X) + c_{mới})$. Nếu $g(Y)$ giảm, đưa Y (hoặc cập nhật lại vị trí của Y) vào frontier và tiếp tục cho A* chạy tiếp.

e) Chi phí tăng n; cạnh $X \to Y$ không thuộc đường tối ưu cũ và chưa được khám phá

- **Chi phí tối ưu có thay đổi không:** Không thay đổi.

- **Cách cập nhật:** Cạnh này chưa từng được khám phá mà chi phí lại tăng lên thì càng không có cơ hội cạnh tranh với đường tối ưu hiện tại. Giữ nguyên toàn bộ kết quả cũ.

f) Chi phí giảm n; cạnh $X \to Y$ không thuộc đường tối ưu cũ và chưa được khám phá

- **Chi phí tối ưu có thay đổi không:** Có thể thay đổi (nếu chi phí cạnh này giảm đủ sâu để tạo ra một đường đi mới ngắn hơn đường tối ưu cũ).
- **Cách cập nhật:** Đưa cạnh này với trọng số mới vào đồ thị. Nếu X đã từng được mở rộng, thêm Y vào frontier với độ ưu tiên $f(Y) = g(X) + c_{mới} + h(Y)$ và tiếp tục chạy thuật toán A*.

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

- A: act(percept) → f_A()
- B: act(percept) → f_B(percept)
- C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Môi trường: Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh

- **Các lớp tác tử có thể hành động hợp lý hoàn hảo:** B và C
- **Giải thích:** Vì môi trường quan sát đầy đủ (Fully observable) và tĩnh (Static), percept tại thời điểm hiện tại chứa toàn bộ thông tin về trạng thái của thế giới. Tác tử B chỉ cần ánh xạ trạng thái hiện tại thành hành động tối ưu mà không cần nhớ quá khứ. Tác tử C chứa nhiều thông tin hơn B nên đương nhiên cũng làm được. Tác tử A không thể vì không nhận percept nên không biết trạng thái hiện tại.

b) Môi trường: Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh

- **Các lớp tác tử có thể hành động hợp lý hoàn hảo: Chỉ có lớp** C
- **Giải thích:** Vì môi trường chỉ quan sát được một phần (Partially observable), một quan sát đơn lẻ tại thời điểm hiện tại không đủ để xác định trạng thái thực của thế giới. Tác tử bắt buộc phải có trạng thái nội bộ / lưu lại lịch sử quan sát để suy diễn trạng thái niềm tin (belief state). Do đó chỉ có tác tử C mới có thể hành động hợp lý hoàn hảo.

c) Môi trường: Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh

- **Các lớp tác tử có thể hành động hợp lý hoàn hảo:** Chỉ có lớp C
- **Giải thích:** Tương tự như trên, việc quan sát một phần đòi hỏi việc theo dõi lịch sử để ước lượng phân phối xác suất trạng thái. Hơn nữa, do môi trường ngẫu nhiên, các kết quả của hành động trong quá khứ cần được tổng hợp để đưa ra kỳ vọng tiện ích tối đa.

d) Đúng hay sai: "Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý"?

- **Đáp án:** ĐÚNG
- **Ví dụ chứng minh:**

- Xét một môi trường tầm thường (trivial environment) mà ở đó mọi hành động của tác tử đều dẫn đến cùng một mức độ phần thưởng/tiện ích như nhau (hoặc môi trường chỉ có duy nhất 1 hành động hợp lệ, ví dụ hành động NoOp).
- Trong môi trường này, dù tác tử chọn hành động nào (ngẫu nhiên, cố định, hay dựa trên percept), tiện ích kỳ vọng nhận được luôn đạt giá trị cực đại có thể. Theo định nghĩa về tính hợp lý (hành động để tối đa hóa tiện ích kỳ vọng), mọi tác tử trong môi trường này đều hành động hợp lý hoàn hảo.